

EF Core .NET 8+ Cheat Sheet - Data Annotations vs Fluent API

Common model mapping attributes and the corresponding Fluent API expressions. Examples use Doctor / DoctorDetail style entities where possible.

Entity and property mapping

Data Annotation	Fluent API equivalent (.NET 8+ / EF Core)	Notes
<code>[Table("Doctors", Schema = "clinic")]</code>	<code>modelBuilder.Entity<Doctor>() .ToTable("Doctors", "clinic");</code>	Maps the entity to a table and optional schema.
<code>[Key] public int Id { get; set; }</code>	<code>modelBuilder.Entity<Doctor>() .HasKey(d => d.Id);</code>	Configures the primary key.
<code>[PrimaryKey(nameof(BranchId), nameof(DoctorId))] public class DoctorAssignment { ... }</code>	<code>modelBuilder.Entity<DoctorAssignment>() .HasKey(x => new { x.BranchId, x.DoctorId });</code>	Class-level attribute for composite keys in modern EF Core.
<code>[Keyless] public class DoctorReport { ... }</code>	<code>modelBuilder.Entity<DoctorReport>() .HasNoKey();</code>	For views, raw SQL result types, and read-only query types.
<code>[Column("doctor_name", TypeName = "nvarchar(100)", Order = 2)] public string Name { get; set; }</code>	<code>modelBuilder.Entity<Doctor>() .Property(d => d.Name) .HasColumnName("doctor_name") .HasColumnType("nvarchar(100)") .HasColumnOrder(2);</code>	Column name, type, and order can be configured fluently.
<code>[MaxLength(100)] public string Name { get; set; }</code>	<code>modelBuilder.Entity<Doctor>() .Property(d => d.Name) .HasMaxLength(100);</code>	Limits maximum length for string/array columns.
<code>[StringLength(100)] public string Name { get; set; }</code>	<code>modelBuilder.Entity<Doctor>() .Property(d => d.Name) .HasMaxLength(100);</code>	EF Core uses the maximum length for schema; minimum length is validation logic, not schema.
<code>[Required] public string Name { get; set; }</code>	<code>modelBuilder.Entity<Doctor>() .Property(d => d.Name) .IsRequired();</code>	For nullable reference types, EF Core may infer required/optional from C# nullability.
<code>[Unicode(false)] public string Code { get; set; }</code>	<code>modelBuilder.Entity<Doctor>() .Property(d => d.Code) .IsUnicode(false);</code>	Useful for varchar vs nvarchar on relational providers that support it.
<code>[Precision(9, 2)] public decimal Fee { get; set; }</code>	<code>modelBuilder.Entity<Doctor>() .Property(d => d.Fee) .HasPrecision(9, 2);</code>	Controls decimal/date-time precision where supported by the provider.

EF Core .NET 8+ Cheat Sheet - Data Annotations vs Fluent API

Generated values, concurrency, indexes, comments, and ignored/complex types.

Generated values, concurrency, indexes, and ignored mapping

Data Annotation	Fluent API equivalent (.NET 8+ / EF Core)	Notes
<code>[DatabaseGenerated(DatabaseGeneratedOption.Identity)]</code> <code>public int Id { get; set; }</code>	<code>modelBuilder.Entity<Doctor>() .Property(d => d.Id) .ValueGeneratedOnAdd();</code>	Common for identity columns and generated primary keys.
<code>[DatabaseGenerated(DatabaseGeneratedOption.Computed)]</code> <code>public DateTime UpdatedAt { get; set; }</code>	<code>modelBuilder.Entity<Doctor>() .Property(d => d.UpdatedAt) .ValueGeneratedOnAddOrUpdate(); // For real computed SQL: .Property(d => d.FullName) .HasComputedColumnSql("[FirstName] + ' ' + [LastName]");</code>	Computed means the database generates the value. Use <code>HasComputedColumnSql</code> when the column expression is known.
<code>[DatabaseGenerated(DatabaseGeneratedOption.None)]</code> <code>public int Id { get; set; }</code>	<code>modelBuilder.Entity<Doctor>() .Property(d => d.Id) .ValueGeneratedNever();</code>	The app must provide the value.
<code>[Timestamp]</code> <code>public byte[] RowVersion { get; set; }</code>	<code>modelBuilder.Entity<Doctor>() .Property(d => d.RowVersion) .IsRowVersion();</code>	Configures a row-version / concurrency token column.
<code>[ConcurrencyCheck]</code> <code>public string Name { get; set; }</code>	<code>modelBuilder.Entity<Doctor>() .Property(d => d.Name) .IsConcurrencyToken();</code>	Uses the property in optimistic concurrency checks.
<code>[Index(nameof(Email), IsUnique = true)]</code> <code>public class Doctor { ... }</code>	<code>modelBuilder.Entity<Doctor>() .HasIndex(d => d.Email) .IsUnique();</code>	<code>IndexAttribute</code> is placed on the entity class, not the property.
<code>[Comment("Doctor display name")]</code> <code>public string Name { get; set; }</code>	<code>modelBuilder.Entity<Doctor>() .Property(d => d.Name) .HasComment("Doctor display name");</code>	Adds database comments where the provider supports them.
<code>[NotMapped]</code> <code>public string DisplayName { get; set; }</code>	<code>modelBuilder.Entity<Doctor>() .Ignore(d => d.DisplayName);</code>	Excludes a property from the EF model.
<code>[NotMapped]</code> <code>public class TemporaryDto { ... }</code>	<code>modelBuilder.Ignore<TemporaryDto>();</code>	Excludes a type from the EF model.
<code>[ComplexType]</code> <code>public class Address { ... }</code> <code>public Address Address { get; set; }</code>	<code>modelBuilder.Entity<Doctor>() .ComplexProperty(d => d.Address);</code>	EF Core 8+ supports complex types. They have no separate key and are stored with the owner.
<code>[Owned]</code> <code>public class DoctorDetail { ... }</code>	<code>modelBuilder.Entity<Doctor>() .OwnsOne(d => d.DoctorDetail);</code>	Owned types are EF Core specific. Useful when the dependent lifecycle belongs to the owner.

EF Core .NET 8+ Cheat Sheet - Data Annotations vs Fluent API

Relationship annotations and Fluent API equivalents. Fluent API is usually clearer for relationships, especially one-to-one.

Relationships

Data Annotation	Fluent API equivalent (.NET 8+ / EF Core)	Notes
<pre>public int DoctorId { get; set; } [ForeignKey(nameof(DoctorId))] public Doctor Doctor { get; set; }</pre>	<pre>modelBuilder.Entity<DoctorDetail>() .HasOne(dd => dd.Doctor) .WithOne(d => d.DoctorDetail) .HasForeignKey<DoctorDetail>(dd => dd.DoctorId);</pre>	This is the direct fix for the one-to-one dependent-side error: DoctorDetail is dependent and owns DoctorId.
<pre>[Required] public int DoctorId { get; set; } public Doctor Doctor { get; set; }</pre>	<pre>modelBuilder.Entity<DoctorDetail>() .HasOne(dd => dd.Doctor) .WithOne(d => d.DoctorDetail) .HasForeignKey<DoctorDetail>(dd => dd.DoctorId) .IsRequired();</pre>	Required relationship is represented by a non-nullable FK. IsRequired makes the relationship explicit.
<pre>public int? DoctorId { get; set; } public Doctor? Doctor { get; set; }</pre>	<pre>modelBuilder.Entity<DoctorDetail>() .HasOne(dd => dd.Doctor) .WithOne(d => d.DoctorDetail) .HasForeignKey<DoctorDetail>(dd => dd.DoctorId) .IsRequired(false);</pre>	Nullable FK means optional relationship.
<pre>[InverseProperty(nameof(Doctor.Appointments))] public Doctor Doctor { get; set; }</pre>	<pre>modelBuilder.Entity<Appointment>() .HasOne(a => a.Doctor) .WithMany(d => d.Appointments) .HasForeignKey(a => a.DoctorId);</pre>	Use InverseProperty when there are multiple navigations between the same two entity types.
<pre>[ForeignKey(nameof(DoctorId))] public Doctor Doctor { get; set; } public ICollection<Appointment> Appointments { get; set; } }</pre>	<pre>modelBuilder.Entity<Appointment>() .HasOne(a => a.Doctor) .WithMany(d => d.Appointments) .HasForeignKey(a => a.DoctorId);</pre>	Typical one-to-many relationship: one Doctor has many Appointments.
No good Data Annotation equivalent for full delete behavior.	<pre>modelBuilder.Entity<Appointment>() .HasOne(a => a.Doctor) .WithMany(d => d.Appointments) .HasForeignKey(a => a.DoctorId) .OnDelete(DeleteBehavior.Restrict);</pre>	Cascade/restrict/no-action behavior should be configured with Fluent API.
No Data Annotation equivalent for alternate/principal key mapping.	<pre>modelBuilder.Entity<Doctor>() .HasAlternateKey(d => d.LicenseNumber); modelBuilder.Entity<DoctorDetail>() .HasOne(dd => dd.Doctor) .WithOne(d => d.DoctorDetail) .HasPrincipalKey<Doctor>(d => d.LicenseNumber) .HasForeignKey<DoctorDetail>(dd => dd.DoctorLicenseNumber);</pre>	Use Fluent API when the FK references something other than the principal primary key.
No complete Data Annotation equivalent for custom many-to-many join table.	<pre>modelBuilder.Entity<Student>() .HasMany(s => s.Courses) .WithMany(c => c.Students) .UsingEntity("StudentCourse");</pre>	Skip navigations can work by convention, but custom join table details are Fluent API work.
Old EF6 style: HasRequired(...).WithRequiredPrincipal(...)	EF Core style: <pre>modelBuilder.Entity<Doctor>() .HasOne(d => d.DoctorDetail) .WithOne(dd => dd.Doctor) .HasForeignKey<DoctorDetail>(dd => dd.DoctorId) .IsRequired();</pre>	HasRequired, WithRequiredPrincipal, WithRequiredDependent are EF6 APIs. Use HasOne/WithOne/HasForeignKey in EF Core.

Quick rule: Data Annotations are convenient for simple mapping. Fluent API is safer for relationships, provider-specific behavior, delete rules, alternate keys, and anything that must be unambiguous.